Deep Learning Primer

Neural networks, training, and frameworks

Day 1 - Session 2

KavinSchool

What We'll Cover

- What deep learning is and when it helps
- The structure of a neural network
- How networks learn: loss, gradients, and backpropagation
- Demos: loss functions and gradient descent
- TensorFlow and PyTorch
- Lab 1.2: image classification

What Is Deep Learning?

Neural networks with many layers

KavinSchool

Deep Learning in One Line

Deep learning uses neural networks with many layers to learn features directly from raw data.

- Classic ML: a human designs the features, the model learns the mapping
- Deep learning: the network learns the features and the mapping together

This is why it shines on images, audio, and text.

> In practice: a bank reads millions of scanned checks by letting the network learn each digit and letter shape on its own, instead of hand-coding a rule for every handwriting style.

Data Types and Volumes

Data type	Example	Common architecture
Images	Photos, scans	Convolutional neural networks (CNNs)
Text	Reviews, chat	Transformers
Audio	Speech, music	CNNs, transformers
Tabular	Spreadsheets	Trees often win

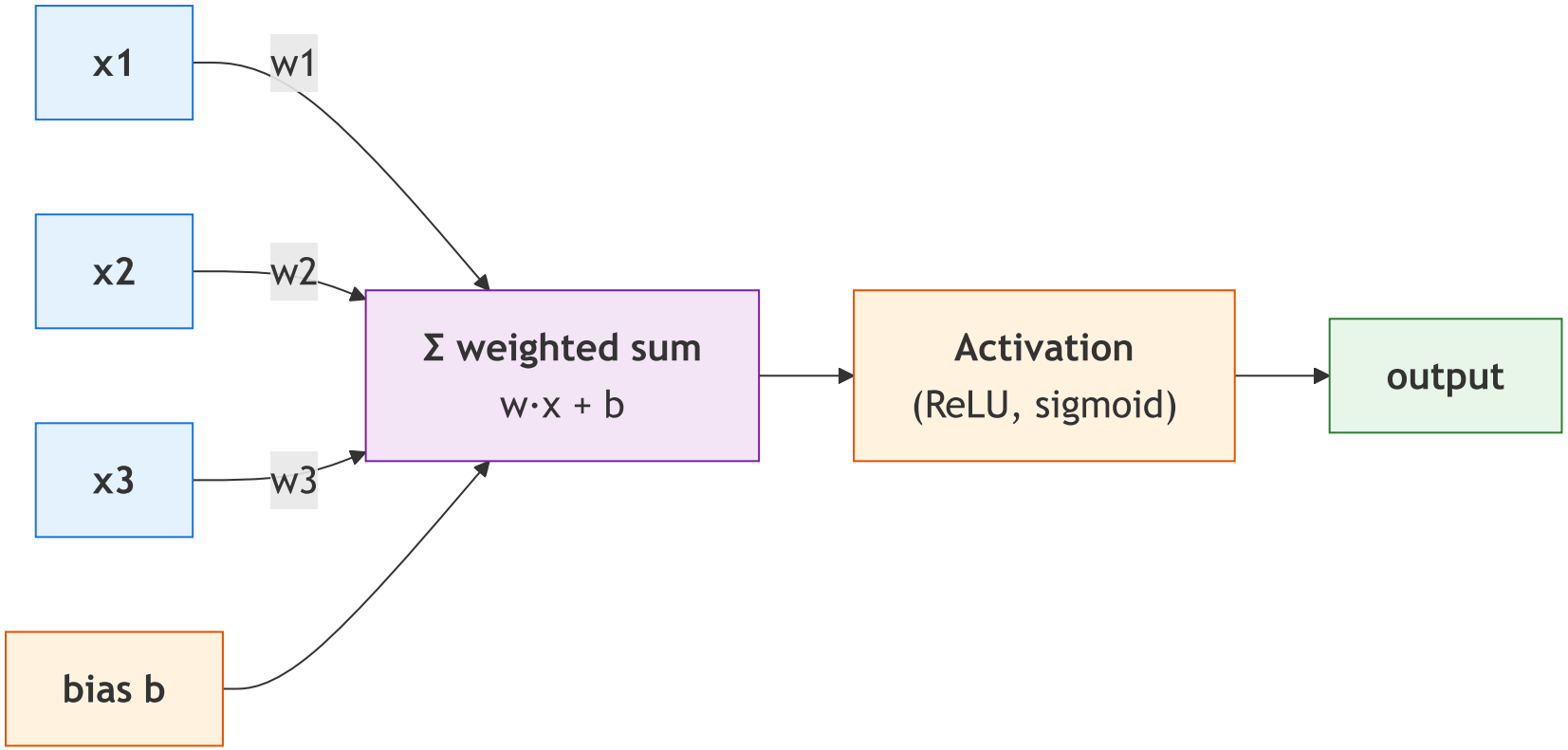
Deep learning usually needs more data than classic ML to pay off.

Inside a Neural Network

Neurons, layers, and activations

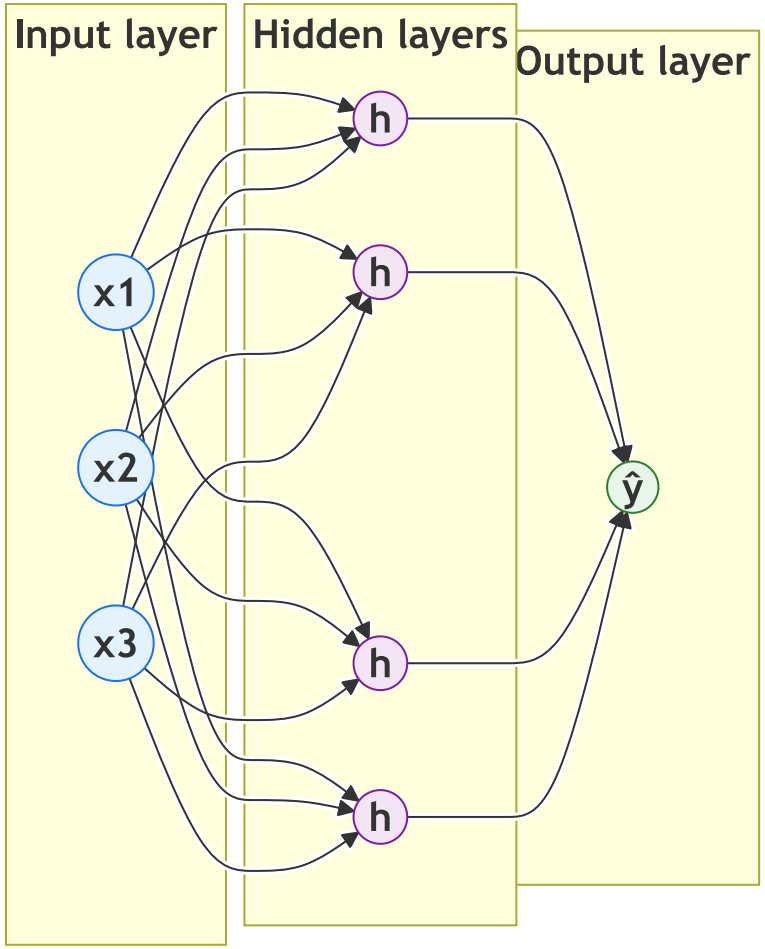
KavinSchool

A Single Neuron



Each neuron multiplies inputs by weights, adds a bias, then applies an activation function.

Stacking Neurons Into Layers



Why Activation Functions Matter

Without a non-linear activation, stacking layers is just one big linear function.

- **ReLU (rectified linear unit):** fast, the default for hidden layers
- **Sigmoid:** squashes to 0-1, used for probabilities
- **Softmax:** turns scores into a probability distribution over classes

Non-linearity is what lets networks model complex patterns.

Common Architectures

Architecture	Best at	Example use
Feedforward (MLP)	Tabular, simple tasks	Credit scoring
CNN	Grid data like images	Digit recognition
RNN / LSTM	Sequences	Older text models
Transformer	Sequences at scale	Modern LLMs

MLP: multilayer perceptron. RNN: recurrent neural network. LSTM: long short-term memory. LLM: large language model.

Intuit Scenario: QuickBooks and TurboTax

Intuit runs deep learning on everyday financial data.

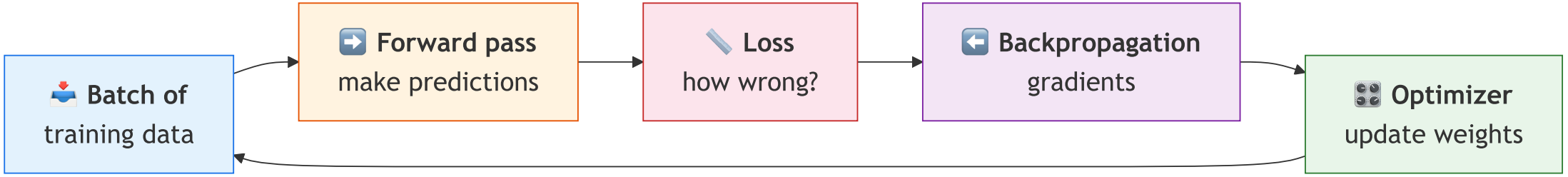
- **QuickBooks (tabular):** a feedforward network reads each bank transaction and predicts its category, so a "SHELL #4021" charge lands under Fuel
- **TurboTax (images):** a convolutional network reads a photo of a W-2 and pulls out wages, employer, and withholding
- **Learned from data:** both improve from millions of past entries that customers confirmed or corrected

How Networks Learn

Loss, gradients, and backpropagation

KavinSchool

The Training Loop



Every step: predict, measure error, compute gradients, nudge the weights.

The Loss Function

The loss is a single number that says how wrong the model is right now.

- **Cross-entropy:** classification (predicting a category)
- **Mean squared error:** regression (predicting a number)

Training is the search for weights that make the loss small.

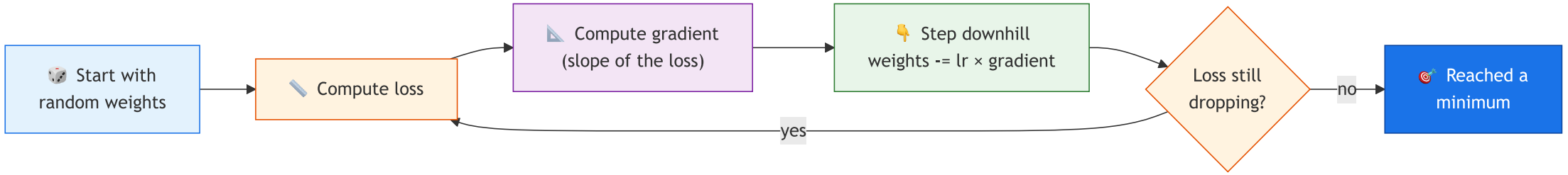
Demo: Loss Functions

Run `day1/demos/demo-loss-functions.py`.

- Plot mean squared error, which grows with the squared error for regression
- Plot cross-entropy, which punishes a confident wrong answer for classification
- See how each loss reacts as the prediction moves away from the target

> A picture of why the loss you pick has to match the task.

Gradient Descent



Demo: Gradient Descent

Run `day1/demos/demo-gradient-descent.py`.

- Start from a random weight and step downhill on a simple loss
- Watch the loss drop as the weight moves toward the minimum
- Change the learning rate to see the step size grow or shrink

> The numbers behind the diagram, moving step by step.

Learning Rate and Optimizers

- **Learning rate:** how big each downhill step is
  - Too big: the loss bounces and diverges
  - Too small: training crawls
- **Optimizers:** SGD (stochastic gradient descent) is the baseline; Adam adapts the step size per weight and is the common default

Backpropagation is the algorithm that computes the gradients efficiently.

Training a Real Model

A fraud model at a payments company learns with the same loop:

- **Forward pass:** score a batch of transactions as fraud or not fraud
- **Loss:** cross-entropy measures how far each score is from the truth
- **Gradients:** backpropagation finds which weights caused the errors
- **Update:** the optimizer nudges those weights so the next scores improve

After many passes over past transactions, the model flags new fraud it was never shown.

Demo: TensorFlow Playground

Open playground.tensorflow.org and watch a network learn live.

- Add hidden layers and neurons, see the decision boundary change
- Raise the learning rate until training becomes unstable
- Switch datasets and note which need more capacity

> A hands-on feel for capacity, learning rate, and overfitting.

Deep Learning Frameworks

TensorFlow and PyTorch

KavinSchool

TensorFlow vs PyTorch

	PyTorch	TensorFlow
Feel	Pythonic, flexible	Production tooling
Research	Dominant	Common
Deployment	TorchServe, ExecuTorch	TensorFlow Serving, TensorFlow Lite
In this course	Primary	Shown as an option

Both build the same networks. We use PyTorch for the labs.

A Model in PyTorch

```
import torch.nn as nn

model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(28 * 28, 128),
    nn.ReLU(),
    nn.Linear(128, 10),
)
```

A few lines define a full image classifier. The training loop fills in the weights.

Breakout Lab 1.2

Image classification with PyTorch

KavinSchool

Lab 1.2: Image Classification

Open `day1/lab1_2_image_clf/start.ipynb` in Colab.

- Load a small image dataset (FashionMNIST)
- Build a neural network with PyTorch
- Train it and watch the loss drop
- Evaluate accuracy on the test set

> Stretch goal: add a hidden layer or try a convolutional layer and compare accuracy.

Key Takeaways

1. Deep learning learns features and predictions together from raw data
2. A neuron is a weighted sum plus a non-linear activation
3. Networks learn by minimizing a loss with gradient descent
4. Backpropagation computes the gradients; the optimizer applies them
5. PyTorch and TensorFlow build the same models; we use PyTorch

Questions?

Next: Overview of Generative AI

Kahoot, then lunch

KavinSchool